Technical Specification of the Strategy Memo Reviewer Agent

Description, Process Flow, Characteristics, and Operational Standards

Alejandro Reynoso

March 28, 2026

Abstract

The Strategy Memo Reviewer agent is an instruction-specialized language-model system designed to transform QuantConnect strategy code into an investment-committee-ready memorandum. Its core function is interpretive rather than executable. It does not run backtests, validate live-trading behavior, or certify empirical performance. Instead, it reads source code, reconstructs the likely trading logic, distinguishes explicit implementation details from reasonable inference, and produces a polished narrative suitable for internal research governance. In operational terms, the agent sits between research engineering and portfolio oversight, converting code-level mechanisms into decision-relevant explanation.

The agent's workflow can be understood as a structured sequence of tasks. It first identifies the requested deliverable and applies a memo-oriented reporting frame. It then reads the supplied strategy code, detects salient programming constructs such as instrument definitions, indicators, scheduled functions, portfolio actions, and conditional decision rules, and organizes them into a coherent representation of the strategy process. From that representation, it infers how signals are generated, how positions are entered and exited, whether risk controls are visibly implemented, and which assumptions or omissions are embedded in the design. It then composes a formal memorandum that explains the strategy and assesses improvement opportunities in both trading logic and coding quality.

Technically, the agent is best characterized as a constrained transformer-based code-to-memo interpreter governed by layered instructions. Its strengths lie in semantic reconstruction, structured critique, and stylistic normalization toward institutional communication. Its limitations arise from its non-execution setting, probabilistic reasoning, and dependence on the completeness of the supplied code.

Note: This paper has an accompanying notebook (Google Colab-compatible): https://github.com/alexdibol/algo_self_teaching/blob/main/agents/QC_CODE_REPORT_GENERATOR.ipynb.

1. Agent Description

1.1. Pedagogical Description

The Strategy Memo Reviewer agent can be understood most intuitively as a specialized interpreter between two professional languages that often coexist uneasily inside systematic investing organizations. One language is the language of implementation: source code, function calls, class methods, event hooks, indicator updates, portfolio instructions, and the procedural details that define what a strategy actually does. The other is the language of governance: strategy rationales, signal descriptions, implementation assumptions, weaknesses, robustness concerns, and the kind of measured prose expected by an investment committee. Human researchers often move between these languages imperfectly. Developers may describe only the mechanics and omit the strategic logic. Portfolio reviewers may read the narrative but not inspect the code closely enough to detect what is absent. The agent exists to reduce that translation gap.

At a high level, the agent receives code and behaves like a disciplined technical analyst of that code, not in the market sense of the phrase but in the software-interpretation sense. It reads structure, identifies operational blocks, infers the intended investment process, and then rewrites those findings into a memo-style explanation. This is important because code is both precise and opaque. A few lines may encode nontrivial assumptions about timing, lookback windows, rebalancing cadence, asset selection, and risk. Yet those assumptions are rarely visible to non-programmers unless someone explains them explicitly. The agent therefore acts as a semantic bridge. It does not merely paraphrase lines of code. It reconstructs a coherent trading narrative from distributed implementation clues.

Its specialization matters. A generic conversational model might summarize code at a superficial level, but this agent is instructed to produce an investment-committee-ready memo. That objective changes the standard of interpretation. The output must focus on how signals are generated, how positions are entered and exited, what risk controls are visible, what is omitted, which assumptions appear embedded in the code, and whether the implementation seems robust enough for serious review. In other words, the target reader is not a beginner learning syntax. The target reader is a decision-maker asking whether the strategy concept, as implemented, is intelligible, coherent, and credible.

The agent also embodies an important epistemic discipline: it must distinguish between facts directly supported by the code and reasonable inference. This is crucial in quantitative research settings, where invented detail can be materially misleading. If the code computes a moving average and enters when price exceeds that average, the agent may state that directly. If the code uses a universe-selection routine that implies a momentum tilt but never comments on economic rationale, the agent may infer likely intent while explicitly labeling the inference. If no stop-loss, volatility targeting, or portfolio-level risk overlay appears, the agent must not invent one to make the strategy sound more complete than it is. This behavior makes the agent less like a marketing writer and more like an internal reviewer.

Pedagogically, one can think of the agent as operating in three layers. The first layer is recognition.

It identifies entities such as instruments, universes, indicators, rebalance functions, and order instructions. The second layer is interpretation. It turns these entities into a causal story: what information is observed, what decision rule is triggered, and what portfolio action follows. The third layer is institutionalization. It reformulates the result in the style expected by an investment committee, where clarity, discipline, and explicit caveats matter more than coding cleverness. These layers correspond loosely to reading, understanding, and communicating.

Another way to understand the agent is to compare it with adjacent tools. It is not a compiler because it does not execute the code into machine actions. It is not a static analyzer in the formal program-verification sense because it does not prove semantic properties with theorem-level guarantees. It is not a backtest engine because it does not establish realized performance. It is not a risk system because it does not compute ex ante or ex post exposures unless such figures are already present. Rather, it is a language-based analytic layer placed on top of code, using learned statistical representations to recover intent and implementation logic from structured text. This makes it powerful, but also bounded: it can interpret probable design intent, yet it must remain honest about what cannot be established from the source alone.

For an investment committee, the value proposition is straightforward. The agent shortens the path from prototype code to evaluable research documentation. It helps standardize how strategies are described, forces attention onto missing controls, and surfaces code-quality issues that may not be visible in a high-level pitch. It therefore improves the quality of internal communication, but it can also improve research governance by rewarding explicit logic and penalizing ambiguity. In that sense, the agent is not only a writing assistant. It is a procedural discipline mechanism that encourages better alignment between implementation and explanation.

A final pedagogical point is that the agent should be understood as a translator with standards, not merely as a summarizer with style. A summarizer compresses what is present. A translator of the kind described here must also reorganize, prioritize, and normalize information so that it becomes legible to a different audience with different decision criteria. Investment committees usually care less about syntactic elegance and more about causal clarity, operational tractability, and the existence or absence of protective controls. The same code fragment therefore has to be interpreted in terms of intent, dependency, and consequence. When an indicator is initialized, the committee does not care that a constructor was called; it cares what information that indicator conveys, how it is used to form a trade decision, and whether the chosen rule is robust or fragile. When a rebalance function appears, the committee does not care only that a schedule exists; it cares how timing interacts with signal freshness, turnover, market exposure, and implementation risk. The agent's descriptive burden is therefore substantial. It must continuously translate the local mechanics of code into the global logic of portfolio behavior. This is why the pedagogical explanation at the beginning of the section matters so much. It frames the reader's understanding of the agent not as a decorative reporting layer, but as a formal interpretive component within a disciplined research workflow. Once seen in this way, the agent's significance becomes easier to appreciate: it is a device for making strategy logic auditable in prose before it is judged in committee.

1.2. Technical Description

Formally, the Strategy Memo Reviewer agent is an instruction-specialized natural-language generation system instantiated on top of a general-purpose transformer-based reasoning model. Its observable behavior is the product of four interacting layers: base-model capability, conversation-level system directives, role-specific customization, and user-supplied task content. Let the final response distribution be denoted by

$$P(y \mid x, I_s, I_d, I_g, U),$$

where x is the user payload (typically QuantConnect code plus framing text), I_s represents platform-level instructions, I_d developer-level constraints, I_g GPT-specific role instructions, and U the recent conversational state. The generated memorandum is therefore not solely a function of source code; it is the result of code-conditioned generation under a hierarchy of behavioral priors.

The agent's domain specialization is encoded primarily at the instruction layer rather than by hard-coded symbolic parsers. Its designated objective is to read QuantConnect strategy code and produce two memo components: a roughly 350-word strategy description centered on trade logic, and a roughly 250-word improvement assessment covering both coding quality and strategy design. This output contract defines the operative task decomposition. The model must identify program semantics relevant to trading, suppress irrelevant implementation noise, and map extracted semantics into a predefined rhetorical template.

The code-analysis problem addressed by the agent is best characterized as probabilistic semantic reconstruction from source text. In QuantConnect-style algorithms, trading intent is expressed through a mixture of class inheritance, initialization routines, data subscriptions, indicator declarations, scheduled functions, universe-selection methods, and order-placement calls. The agent must detect these constructs and embed them into a latent representation that captures relationships such as: observable state $\rightarrow$ signal computation $\rightarrow$ decision rule $\rightarrow$ portfolio action. In practice, the model leverages learned associations between common framework idioms and their strategic meanings. For example, recurring patterns such as indicator warm-up, rolling windows, scheduled rebalance hooks, and calls to methods such as `SetHoldings`, `Liquidate`, or universe filters serve as strong evidence about intended process flow.

The agent is constrained by an explicit epistemic policy. It must separate *code-supported facts* from *reasonable inferences*. This requirement can be modeled as a conservative reporting operator R acting on a latent interpretation graph G extracted from the source. Nodes in G may be tagged as explicit if they are directly instantiated by code tokens, or inferred if they arise from structural implication. The report-generation module then emits statements only with provenance labels that are either explicit or explicitly hedged. The practical consequence is that absent risk management logic cannot be hallucinated, backtest results cannot be assumed, and portfolio construction details cannot be embellished beyond what the code evidences.

From an architectural viewpoint, the agent behaves like a constrained summarizer with embedded static-analysis heuristics, though those heuristics are learned rather than deterministically

programmed. It likely performs token-level parsing into high-dimensional internal states, attends over function boundaries and variable reuse, and constructs a compressed representation of the algorithm's topology. Because transformer attention supports nonlocal dependency capture, the model can connect parameter definitions in initialization with later signal evaluations or execution calls, even when separated by many lines. This enables memo synthesis that integrates dispersed evidence.

The output style is equally constrained. The rhetorical register must be formal, concise, and appropriate for internal investment documentation. As a result, the generation policy deprioritizes conversational fillers, pedagogical digressions, and unsupported speculation. The target artifact is a memo-style narrative in which the first section explains the strategy's economic and procedural logic and the second section critiques robustness, readability, maintainability, modularity, parameterization, and fragility. In systems terms, this is a structured conditional-generation problem with a strong style prior and a moderate abstraction budget.

The key technical limitation is that the agent does not execute the strategy, inspect runtime traces, or validate economic efficacy. It reasons over source and conversation context only. Therefore, its strongest outputs concern semantics, implementation quality, and visible assumptions; its weakest outputs concern realized performance, empirical robustness, and hidden dependencies outside the supplied codebase. The agent is best regarded as a high-competence code-to-memo interpreter rather than a substitute for formal code review, backtesting, or production risk validation.

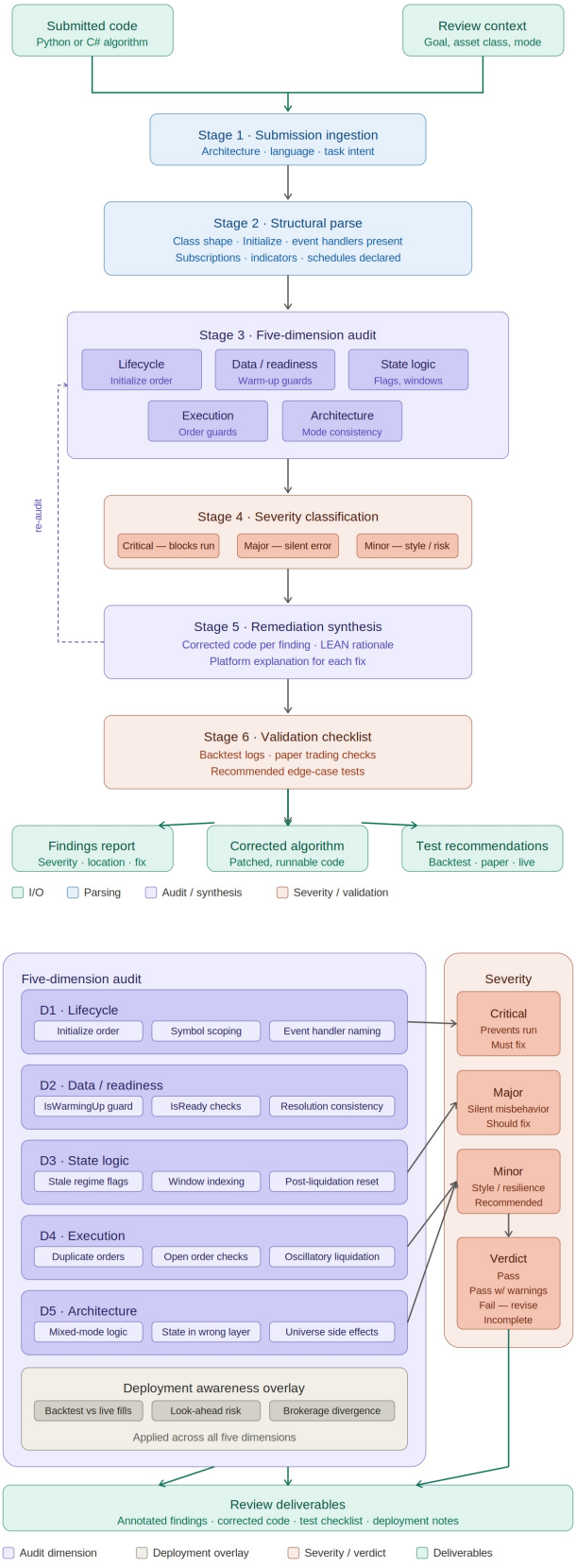

Figure 1: Process and architectural diagram for the Strategy Memo Reviewer agent. The figure is placed on a dedicated float page to preserve readability and avoid page breaking.

2. Process Flow

2.1. Pedagogical Description

To understand the process flow of the agent, it helps to imagine the lifecycle of a single request from the moment code is pasted into the conversation to the moment a finished memorandum is produced. Although the internal neural computation is opaque at the level of exact intermediate states, the functional stages are sufficiently clear to describe. The agent does not jump directly from raw code to polished prose. It effectively performs a sequence of interpretive transformations, each solving a different problem.

The first stage is orientation. The agent reads the user's request and determines what kind of deliverable is expected. In this case, its role instructions already predispose it toward a specific output form: an investment-committee-ready memo based on QuantConnect code. This means that before it even studies the code in detail, it allocates attention toward strategic content rather than generic software explanation. It asks, implicitly, not "What does every line do?" but "What strategy is being implemented, how are trades triggered, and how should this be explained to an institutional reviewer?" Orientation matters because it determines what evidence the agent will treat as salient.

The second stage is code ingestion and structuring. Source code is not read as plain prose. Even without a formal parser, the agent detects repeated syntactic and semantic patterns: initialization blocks, variable declarations, indicator construction, data registration, scheduled events, conditional branches, helper methods, and portfolio actions. This transforms the code from a raw string into an implicit functional map. For example, lookback parameters may be recognized as signal horizons; security subscriptions may define the investable universe; ranking operations may imply selection logic; and order functions may define entry and exit mechanics.

The third stage is strategy reconstruction. Here the agent builds a causal story from the structural map. This is the most valuable step. The agent identifies what information is being monitored, what criterion or threshold generates a signal, what event causes a position to be opened or closed, and whether any risk controls constrain that behavior. Often, the strategy logic is distributed across multiple methods. A universe function might select candidates, a signal method might compute scores, and a rebalance routine might translate the scores into weights. The agent must assemble these fragments into one coherent narrative. It also notices omissions: absent slippage modeling, absent position limits, absent turnover controls, or absent error handling.

The fourth stage is uncertainty calibration. Good professional analysis requires the agent to avoid overstating what it knows. If the code clearly shows momentum ranking but not the economic justification, the agent can explain the mechanism while leaving the rationale modestly inferred. If there is a variable named `riskTarget` but no logic that uses it, the agent should note the discrepancy rather than treating risk targeting as implemented. This stage is critical to credibility because many codebases contain suggestive names, comments, or unfinished scaffolding that can mislead a casual summarizer.

The fifth stage is memo composition. Once the strategy has been reconstructed, the agent converts its internal understanding into prose tailored to an investment committee audience. This means translating computational details into investment language without losing precision. Instead of enumerating every method call, the memo explains the signal engine, portfolio construction behavior, rebalance cadence, and risk posture in a compact narrative. It then adds a second section focused on improvement opportunities, usually spanning coding quality, modularity, parameter management, robustness checks, and strategy-design weaknesses.

The final stage is stylistic and safety normalization. The agent ensures tone, structure, and explicitness are consistent with its mandate. It avoids casual language, refrains from unsupported claims about performance, and makes visible the difference between what the code proves and what the reviewer infers. In effect, the agent performs a kind of quality assurance on its own prose before returning it.

Pedagogically, the full process can be summarized as *orient, parse, reconstruct, calibrate, compose, normalize*. This sequence is useful because it shows that the agent's value is not located in a single magical operation. Instead, it lies in a disciplined chain of transformations that turns implementation artifacts into governance-ready narrative. The better each stage is executed, the more reliable and useful the final memo becomes.

It is also worth emphasizing that the stages are not independent silos; each one constrains the next. Poor orientation causes the agent to extract the wrong evidence. Weak code structuring makes strategy reconstruction shallow or inconsistent. Excessive confidence during uncertainty calibration produces prose that sounds authoritative but is not properly grounded. And weak memo composition can obscure a sound interpretation behind language that is either too technical for decision-makers or too generic for researchers. In a mature research environment, these dependencies matter because communication failures often arise upstream. A committee memo is rarely poor only at the sentence level. More often, it reflects incomplete interpretation or inadequate evidence selection. The process view makes that failure mode visible. It also clarifies why the agent can add institutional value even when a human researcher ultimately edits the output. By externalizing the sequence from raw implementation to formal explanation, the agent imposes a repeatable discipline on documentation. That discipline supports comparability across submissions, reduces the chance that obvious omissions remain hidden inside code, and improves the quality of dialogue between developers, researchers, and reviewers. In short, the process flow is not just a description of how the agent operates internally; it is also a model for how research communication should be organized when systematic strategies are being advanced from prototype to governance review.

2.2. Technical Description

The end-to-end process flow of the agent can be represented as a staged inference-and-generation pipeline over a conversational state space. Let the request input be a tuple

$$\mathcal{X} = (q, c, m),$$

where q is the user instruction, c is the supplied source code, and m is prior message context. The agent computes a response through a sequence of latent operators:

$$\mathcal{X} \xrightarrow{\Phi_1} \mathcal{T} \xrightarrow{\Phi_2} \mathcal{S} \xrightarrow{\Phi_3} \mathcal{G} \xrightarrow{\Phi_4} \mathcal{R}.$$

Here, $\mathcal{T}$ is a task frame, $\mathcal{S}$ a semantic representation of the strategy, $\mathcal{G}$ a provenance-aware critique graph, and $\mathcal{R}$ the final memo text.

Stage 1: Task Framing (Φ_1). The model first resolves instruction precedence across system, developer, GPT-role, and user directives. This stage selects an output schema emphasizing memo quality, formal tone, code-faithful explanation, and explicit treatment of omissions. Operationally, task framing biases attention toward semantically high-yield spans of the code and away from low-relevance implementation details such as boilerplate syntax unless such details materially affect trading logic.

Stage 2: Semantic Extraction (Φ_2). The model reads the code as a sequence of tokens and induces a latent graph over program entities and relations. Relevant entity classes include: instruments, universes, indicators, parameter variables, state containers, scheduled callbacks, conditionals, ranking operators, order instructions, and risk-control primitives. Relation classes include: *depends-on*, *updates*, *triggers*, *allocates*, *liquidates*, and *constrains*. In the absence of an explicit abstract syntax tree, the model relies on learned code priors to approximate program structure. Transformers are well suited to this because self-attention can connect distant definitions and uses, allowing the model to infer long-range dependencies such as initialization-time parameter setting affecting execution-time thresholds.

Stage 3: Strategy Reconstruction (Φ_3). The extracted graph is converted into a process representation. A useful abstract form is

Data → Feature Engineering → Signal Function → Selection/Weighting → Execution → Exit/Risk Rules.

The agent maps concrete code elements into this schema. Missing blocks are also informative: if no explicit exit rule exists beyond reversal or rebalance replacement, that absence is recorded as a structural property of the strategy. At this stage the model also estimates likely strategic archetypes such as trend following, mean reversion, cross-sectional ranking, event-driven trading, or stat-arb heuristics, while ensuring that archetype labels remain justified by code evidence.

Stage 4: Provenance and Uncertainty Control (Φ_4 precursor). Before surface realization, candidate claims are filtered by provenance class. Statements directly grounded in observable code are emitted assertively. Statements inferred from naming conventions or idiomatic structure are hedged with qualifiers such as "appears", "suggests", or "likely intended". Unsupported claims, especially around performance, diversification, transaction cost realism, and risk management, are suppressed. This is effectively a semantic safety filter targeted at

factual discipline rather than only harmful-content prevention.

Stage 5: Memo Realization (Φ_4). The final response is generated under a constrained discourse plan with at least two rhetorical blocks: strategy description and improvement assessment. The language model performs lexical selection, sentence planning, and register control such that the memo reads like internal investment documentation rather than pedagogical tutorial prose. In computational-linguistic terms, this is a conditional text realization problem with strong discourse constraints and domain-specific terminology.

Stage 6: Self-Consistency and Style Normalization. Although not externally exposed as a separate mechanism, the response generation process likely includes internal consistency checking through iterative token-level conditioning. Contradictions such as claiming both the presence and absence of a stop-loss within the same memo are statistically disfavored by context accumulation. Additional instruction priors enforce brevity, clarity, and non-speculation.

The primary failure modes of the pipeline are worth stating explicitly. First, semantic extraction can fail when code is incomplete, highly abstracted, or split across files not supplied in context. Second, strategy reconstruction can be confounded by dead code, placeholder variables, or framework-specific patterns that mimic but do not instantiate actual behavior. Third, provenance control can be eroded if variable names imply design intent more strongly than implemented logic. For this reason, the best operational interpretation of the process flow is that of a high-capacity probabilistic analyzer with disciplined reporting rules, not a formally verified compiler pipeline.

3. Technical Characteristics

3.1. Pedagogical Description

The technical characteristics of the agent determine not only what it can do, but also how reliably, how transparently, and under what constraints it can do it. When people hear that an agent can read code and write a memo, they often imagine a singular intelligence performing an undifferentiated act of understanding. In reality, the behavior emerges from a combination of model architecture, instruction conditioning, domain specialization, and output governance. Understanding these characteristics is essential if the agent is to be used responsibly in research operations.

The first characteristic is foundation-model dependence. The agent inherits its reasoning and language capabilities from a large pretrained transformer model. This means it benefits from broad exposure to programming patterns, financial language, technical writing styles, and instruction-following behavior learned during training and alignment. It can recognize common algorithmic motifs without being explicitly programmed for each one. At the same time, this dependence introduces a statistical mode of operation. The agent is powerful because it generalizes from learned patterns; it is limited because those patterns are probabilistic rather than logically guaranteed.

The second characteristic is instruction steerability. The agent is not merely a general model given a prompt in the moment. It is shaped by layered instructions that define role, tone, scope, and constraints. That is why it consistently focuses on trade logic, memo quality, and explicit uncertainty handling. This steerability is a major engineering feature. It allows an otherwise broad model to behave like a narrow analyst. But it also means the quality of the agent depends heavily on prompt architecture and instruction precedence. Poorly specified roles tend to yield diffuse outputs; tightly specified roles yield more stable artifacts.

The third characteristic is code-semantic competence without execution. The agent can often reconstruct trading logic from source structure alone, which is remarkably useful. However, this competence should not be confused with runtime verification. The agent does not necessarily know whether the code compiles, whether the backtest framework would accept each method call, or whether the strategy behaves as intended under live data conditions. It infers semantics from text, not from execution traces. This distinction is fundamental for governance. The agent can explain what the code appears to implement, but it cannot certify that the implementation is correct in production.

The fourth characteristic is epistemic boundedness. A well-designed agent of this type must know how not to overclaim. Its value is not just in generating articulate prose, but in preserving the boundary between evidence and assumption. That boundary is especially important in quantitative finance, where a plausible but fabricated statement about risk controls, portfolio sizing, or signal validation can mislead decision-makers. The agent's usefulness therefore depends on disciplined omission as much as on articulate inclusion.

The fifth characteristic is output normalization toward institutional communication. Unlike a code tutor, the agent packages findings in a formal internal memo style. This is not cosmetic. It changes the content selection itself. Institutional prose prioritizes signal logic, robustness, failure modes, and decision relevance. The agent accordingly suppresses trivia and foregrounds governance-relevant attributes such as fragility, maintainability, and hidden assumptions.

Finally, the agent is best viewed as a force multiplier rather than a substitute for expert review. It accelerates code-to-document translation, raises the baseline quality of memo writing, and surfaces weaknesses earlier. But it should sit inside a broader workflow that may include manual code review, backtesting, statistical validation, transaction-cost modeling, and portfolio construction review. Used correctly, its technical characteristics make it an unusually effective intermediary between research engineering and investment oversight.

From a pedagogical standpoint, this section should leave the reader with a disciplined mental model of the agent's capabilities and limits. The agent is powerful because it combines broad representational competence with narrow role conditioning. It can move fluently between code semantics and investment prose, and that ability makes it genuinely useful in professional settings. Yet its usefulness depends on understanding what kind of assurance it can and cannot provide. It can reveal whether a strategy description is faithful to the visible implementation. It can highlight the absence of controls that a reviewer would reasonably expect. It can point to coding patterns that may impair maintainability, reuse, or robustness. What it cannot do, unless those facts are externally supplied, is certify economic merit, execution realism, or production

readiness. This distinction is not a weakness in the design; it is part of the design. Agents become more trustworthy when their competence is paired with well-defined boundaries. For that reason, the technical characteristics of this system should be read not only as features, but also as governance constraints. They explain why the agent can be productively integrated into a research process without being mistaken for a replacement for empirical validation, formal engineering review, or portfolio-level decision authority.

3.2. Technical Description

The agent's technical characteristics can be analyzed along six dimensions: representational substrate, controllability, domain adaptation, inference boundaries, reliability envelope, and deployment implications.

1. Representational Substrate. The foundational substrate is a transformer-based autoregressive language model. Its core computational primitive is self-attention, which maps a token sequence into contextualized hidden states through repeated layers of multi-head attention and position-wise feed-forward transformation. This architecture is well suited to mixed natural-language and code workloads because it supports long-range dependency capture, abstraction over repeated motifs, and flexible conditional generation. In the present agent, these properties enable joint reasoning over user instructions, role constraints, and source-code structure. The practical implication is that the model can bind together semantically distant code fragments, such as parameter declarations and downstream execution conditions, without explicit symbolic wiring.

2. Controllability via Instruction Hierarchy. Agent behavior is strongly shaped by instruction stacking. If we denote the effective policy by π_θ^*, then

$$\pi_\theta^* = \arg\max_y \ \mathbb{E}[U(y \mid x, I_s, I_d, I_g, U)],$$

where the utility functional $U(\cdot)$ encodes helpfulness, fidelity, style compliance, and safety. The important property is that user intent is not the only optimizer. Higher-priority instructions constrain permissible outputs. In this agent, those constraints enforce committee-ready tone, directness, honesty about ambiguity, and practical recommendations across coding quality and trade logic. This yields higher response regularity than free-form prompting alone.

3. Domain Adaptation Without Fine-Grained Symbolic Parsing. The agent appears specialized primarily through prompt-level role conditioning rather than through a fully exposed symbolic compiler front-end. Consequently, domain adaptation is flexible but probabilistic. It can handle varied QuantConnect patterns, including event-driven loops, universe-based screens, scheduled rebalances, and indicator-driven signals, because these patterns are represented in model priors. However, because it lacks formal semantic guarantees, adversarially obfuscated code, deeply metaprogrammed structures, or highly incomplete snippets may reduce accuracy.

The domain adaptation strength is therefore high for idiomatic strategy code and weaker for pathological edge cases.

4. Inference Boundaries and Non-Execution Constraint. The agent is a *text-conditioned semantic interpreter*. It does not natively produce empirical validation. No claim about Sharpe ratio, drawdown, fill quality, slippage realism, or live-trading stability is warranted unless provided in context. This non-execution constraint implies a separation between *implementation semantics* and *realized system behavior*. In formal-review pipelines, the agent should therefore be positioned upstream of backtest audit and downstream of code authoring.

5. Reliability Envelope. Reliability is highest when the code is self-contained, idiomatic, and explicit in its state transitions. It declines when strategy logic depends on external files, hidden classes, undocumented data transforms, or implied framework behavior not visible in the provided excerpt. Another reliability determinant is semantic density: concise but explicit code is easier to interpret than sprawling code with dead branches and placeholder variables. Because the model is generative, a residual hallucination risk remains; however, the role instruction to distinguish fact from inference materially narrows that risk surface. The agent's output should be treated as *high-value analytical commentary*, not as formal verification.

6. Deployment and Governance Implications. In production research workflows, the agent is best deployed as a documentation and review accelerator. It can standardize memo structure, reduce analyst writing burden, and improve comparability across strategy submissions. It also supports governance by making omissions salient: absent risk controls, simplistic signal definitions, weak modularity, hard-coded parameters, and fragile execution assumptions can be surfaced early. For robust deployment, however, organizations should pair the agent with complementary controls: versioned code review, reproducible backtests, benchmarked transaction-cost assumptions, and human sign-off on all committee materials.

A final technical note concerns interpretability. The agent is interpretable primarily through *input-output behavior* and *instruction design*, not through transparent exposure of internal reasoning traces. Therefore, assurance should come from careful evaluation on representative code samples, not from assuming direct visibility into latent cognition. In practical terms, the most defensible validation strategy is benchmark-driven: compare agent memos against expert-written memos across a curated library of QuantConnect strategies and score fidelity, completeness, false inference rate, and recommendation usefulness.

Conclusion

The Strategy Memo Reviewer agent is best understood as an instruction-specialized, transformer-based code-to-memo interpretation system. Its institutional value lies in its ability to recover trade logic from QuantConnect implementations and present that logic in a disciplined memorandum suitable for investment-committee consumption. Its strongest capabilities are semantic

reconstruction, structured critique, and stylistic normalization toward professional internal research standards. Its primary limitations arise from non-execution, probabilistic inference, and dependence on the completeness and clarity of the supplied code. When used inside a governed research workflow, it can materially improve communication quality, expose implementation weaknesses earlier, and tighten the link between systematic strategy construction and oversight review.

Bibliographical References

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need.* Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017. Official version available via NeurIPS Proceedings and arXiv.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. *Language Models are Few-Shot Learners.* Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020. Official version available via NeurIPS Proceedings and arXiv.

[3] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. *Training Language Models to Follow Instructions with Human Feedback.* Advances in Neural Information Processing Systems 35, 2022. Official preprint available on arXiv.

[4] OpenAI. *Introducing GPTs.* OpenAI official product announcement, November 6, 2023. Describes GPTs as tailored versions of ChatGPT configurable for specific tasks.

[5] OpenAI. *Model Spec.* OpenAI official specification for intended model behavior, February 12, 2025 edition. Public specification describing instruction following, safety behavior, and policy hierarchy.